ESP32-CAM SECURITY ROBOT

Project Technical Documentation

Platform: ESP32-CAM AI Thinker
Domain: Embedded Systems / Robotics / IoT
Interface: Browser-Based WiFi Control
Category: Security / Surveillance / Autonomous Safety

1. Introduction

The ESP32-CAM Security Robot is a compact, WiFi-connected mobile robot designed to provide surveillance and intrusion detection in indoor environments. Built around the ESP32-CAM AI Thinker module, the robot combines live video streaming, on-device face detection, motorized mobility, and obstacle avoidance into a single, browser-accessible platform.

The core problem it addresses is the need for a low-cost, self-contained surveillance unit that can move, observe, and react intelligently without relying on a cloud backend. All processing — including face detection — occurs directly on the ESP32-CAM microcontroller.

The robot can be directed manually via a browser-based control panel over a local WiFi network, while simultaneously streaming live MJPEG video. If a face is detected during the periodic background scan, an audible buzzer alert is triggered immediately. The ultrasonic sensor ensures the robot halts before colliding with nearby obstacles, providing a basic layer of autonomous safety.

Design Philosophy

The project demonstrates that capable surveillance hardware can be built with affordable off-the-shelf components. The entire system runs on a single $10 ESP32-CAM module with no external server, no cloud subscription, and no proprietary SDK.

2. Features

The following capabilities are implemented in the current firmware revision:

- Live MJPEG video streaming accessible from any browser on the local network
- On-device face detection using the ESP32's built-in AI acceleration (no cloud required)
- Buzzer alert triggered automatically when a face is detected
- Full motor control via L298N driver: forward, backward, left turn, right turn, stop
- Ultrasonic obstacle detection using HC-SR04; the robot refuses to move forward when an obstacle is within the safety threshold
- Browser-based control interface with directional command buttons
- Periodic background face detection running independently of the video stream
- WiFi connectivity using Arduino WiFi stack; IP address printed to Serial on boot

3. System Architecture

The robot is structured around six loosely coupled functional modules that communicate through shared state variables and event-driven HTTP handlers. The diagram below describes the logical relationships between modules.

3.1 Module Overview

Module	Responsibility	Interaction
Camera	Capture frames, init OV2640 sensor	Feeds MJPEG streamer and face detector
Web Server	Handle HTTP requests from browser	Dispatches motor and query commands
MJPEG Streamer	Push continuous frames over HTTP	Reads camera frames, writes to client
Face Detection	Run Haar/NN detection on frame	Writes detection result, triggers buzzer
Motor Control	Drive L298N direction signals	Called by web server command handlers
Ultrasonic Safety	Measure distance to obstacles	Guards forward motion commands
Buzzer Alert	Emit audible alert on detection	Called by face detection module

3.2 Data Flow

On each HTTP control request, the web server validates the obstacle distance before executing any forward motion. All other directional commands (backward, left, right, stop) proceed unconditionally. The face detection task runs in the background at a configurable interval, reading a fresh camera frame and writing a boolean flag. If the flag transitions to true, the buzzer fires.

The MJPEG stream is a long-lived HTTP connection that continuously pushes frames to the browser. It operates independently of the control server and does not block or gate command handling.

4. Hardware Components

4.1 Component Summary

Component	Role	Key Notes
ESP32-CAM AI Thinker	Main MCU + camera + WiFi	OV2640 sensor; onboard PSRAM for frame buffers; GPIO pins shared with flash LED
L298N Motor Driver	Dual H-bridge for 2 DC motors	Logic inputs 3.3V compatible; motor supply separate from logic supply

2x DC Motors	Differential drive locomotion	Driven independently for turns; speed control possible via PWM
HC-SR04	Ultrasonic distance sensing	5V device; ECHO line must be level-shifted to 3.3V for ESP32
Active Buzzer	Audible alert output	Driven directly from GPIO4; requires no additional driver circuit
External Power Source	Supply motors and ESP32	Motors draw transient current; use decoupling capacitors near motor supply
Voltage Divider (1kΩ/2.2kΩ)	Level-shift ECHO signal	Reduces HC-SR04 ECHO from 5V to ~3.3V; protects ESP32 GPIO

4.2 Component Notes

ESP32-CAM AI Thinker

The ESP32-CAM integrates the ESP32-S chip, an OV2640 camera, a microSD slot, and a flash LED on a single compact board. Its PSRAM (4MB pseudostatic RAM) enables large frame buffer allocation necessary for MJPEG streaming and face detection. Note that GPIO0 must be pulled LOW during firmware upload and released afterward for normal operation.

L298N Motor Driver

The L298N is a classic dual H-bridge capable of driving two DC motors independently. Its logic input pins (IN1–IN4) are compatible with 3.3V signals from the ESP32-CAM, making interfacing straightforward. The motor supply (VM) must be provided separately from the logic supply (VSS). The onboard 5V regulator on the L298N breakout can power the ESP32-CAM if the motor supply is 7V or higher; otherwise, supply 5V externally.

HC-SR04 Ultrasonic Sensor

The HC-SR04 operates at 5V and returns a TRIG/ECHO pulse interface. TRIG is driven by the ESP32 at 3.3V, which is sufficient to trigger the sensor. However, the ECHO line returns a 5V signal that must not be applied directly to the ESP32 GPIO — a voltage divider is mandatory.

5. Wiring and Connections

5.1 Pin Assignment Table

ESP32-CAM GPIO	Connected To	Direction
GPIO12	L298N IN1	Output
GPIO13	L298N IN2	Output

GPIO14	L298N IN3	Output
GPIO15	L298N IN4	Output
GPIO4	Buzzer (+)	Output
GPIO2	HC-SR04 TRIG	Output
GPIO16	HC-SR04 ECHO (via divider)	Input
GND	All component GNDs	Common
5V	HC-SR04 VCC, L298N VSS	Power supply

5.2 Voltage Divider for ECHO Line

The HC-SR04 ECHO pin outputs a 5V pulse, which exceeds the 3.3V GPIO input tolerance of the ESP32. A resistive voltage divider must be placed on this line to protect the microcontroller.

Voltage Divider Calculation

Vout = Vin × R2 / (R1 + R2) = 5V × 2200 / (1000 + 2200) ≈ 3.44V — safely within the ESP32 3.3V GPIO input range with a small margin.

Divider Schematic (Text Representation)

```
HC-SR04 ECHO ──[ 1kΩ ]──┬──── GPIO16 (ESP32-CAM)
                        │
                      [ 2.2kΩ ]
                        │
                       GND
```

5.3 Power Wiring Notes

- The ESP32-CAM is powered from a stable 5V supply, regulated down to 3.3V internally.
- The HC-SR04 requires 5V; connect to the same 5V rail.
- All GND connections must share a common reference — ESP32 GND, L298N GND, HC-SR04 GND, and buzzer GND.
- DC motors introduce significant current spikes and electrical noise. Use bulk capacitors (100–470 µF) across the motor supply terminals, and bypass capacitors (100 nF) close to the L298N logic pins.
- Power instability is the most common cause of random ESP32 resets during motor operation. If the system is unreliable, add capacitors or use a dedicated motor power supply separate from the logic supply.

6. Software Structure

The firmware is written using the Arduino framework for ESP32 and organized into the following logical modules within a single sketch file.

Module / Function Group	Description
Camera Initialization	Configures OV2640 settings, allocates PSRAM frame buffer, sets resolution and quality for streaming
Motor Control Functions	moveForward(), moveBackward(), turnLeft(), turnRight(), stopMotors() — drive L298N IN1–IN4 pins
Ultrasonic Functions	measureDistance() — sends TRIG pulse, measures ECHO pulse width, returns distance in cm
Buzzer Functions	alertBuzzer() — drives GPIO4 HIGH for a fixed duration to produce an audible beep
Face Detection Function	detectFace() — grabs a frame, runs the face detection pipeline, returns boolean result
Web Server Handlers	HTTP GET handlers for /move?dir=, /stream, /status — parse query params and dispatch actions
Background Face Check Task	FreeRTOS task or timed loop that calls detectFace() periodically and triggers alertBuzzer()
Main Setup (setup())	Initializes Serial, camera, WiFi, motor pins, buzzer pin, ultrasonic pins, starts web server
Main Loop (loop())	Handles incoming HTTP clients; background task runs independently via timer or RTOS task

7. Code Walkthrough

7.1 Motor Control

Motor direction is controlled by setting pairs of digital outputs on IN1–IN4. The L298N interprets each pair as a direction for one motor channel.

```
void moveForward() {
  digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);  // Motor A forward
  digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);  // Motor B forward
}

void stopMotors() {
  digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
}
```

Left and right turns are achieved by driving one motor forward while stopping or reversing the other, producing differential steering.

7.2 Ultrasonic Safety Guard

Before any forward motion command is executed, the firmware measures the distance to the nearest obstacle. If the reading is below the safety threshold (typically 20 cm), the forward command is rejected and the motors remain stopped.

```
long measureDistance() {
  digitalWrite(TRIG_PIN, LOW);  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  long duration = pulseIn(ECHO_PIN, HIGH, 30000);
  return duration * 0.034 / 2;  // Convert to cm
}
```

```
// Inside forward command handler:
if (measureDistance() < SAFE_DISTANCE_CM) {
  stopMotors();
  server.send(200, "text/plain", "BLOCKED");
  return;
}
moveForward();
```

7.3 Face Detection Path

Face detection is performed using the ESP32-CAM's built-in face detection library (part of the esp32-camera component). A frame is captured from the camera buffer, passed through the detection pipeline, and the result is a count of detected faces.

```
bool detectFace() {
  camera_fb_t *fb = esp_camera_fb_get();
  if (!fb) return false;

  // Run face detection on captured frame
  bool faceFound = (run_face_detection(fb) > 0);

  esp_camera_fb_return(fb);
  return faceFound;
}
```

Important: Detection vs. Recognition

This project performs face detection only — it identifies the presence of a human face in the frame. It does NOT perform face recognition and cannot identify who a specific person is. These are fundamentally different tasks requiring different models and substantially more compute.

7.4 Background Face Checking and Buzzer Alert

A periodic background routine calls detectFace() at a fixed interval (e.g., every 2 seconds). If a face is found, alertBuzzer() is triggered. This runs independently of user commands.

```
// Called periodically (every ~2000 ms):
void backgroundFaceCheck() {
  if (detectFace()) {
    alertBuzzer();
  }
}

void alertBuzzer() {
  digitalWrite(BUZZER_PIN, HIGH);
  delay(500);
  digitalWrite(BUZZER_PIN, LOW);
}
```

7.5 MJPEG Stream Handler

The MJPEG stream is served over a dedicated HTTP endpoint. The handler holds the connection open and continuously pushes multipart JPEG frames to the browser. The Content-Type header uses multipart/x-mixed-replace, which browsers interpret as a live video feed.

```
// Pseudocode — actual implementation uses esp32-camera streaming helpers
void handleStream() {
  WiFiClient client = server.client();
  client.println("HTTP/1.1 200 OK");
  client.println("Content-Type: multipart/x-mixed-replace;
boundary=frame");
  while (client.connected()) {
    camera_fb_t *fb = esp_camera_fb_get();
    client.printf("--frame\r\nContent-Type: image/jpeg\r\n\r\n");
    client.write(fb->buf, fb->len);
    esp_camera_fb_return(fb);
  }
}
```

8. How the Robot Works

The following describes the complete runtime behavior from power-on to active operation.

Stage	Action	Notes
1. Boot	ESP32 executes bootloader, loads firmware from flash	GPIO0 must be HIGH (not in flash mode)
2. Serial Init	Serial port started at 115200 baud for debug output	Useful for IP address and error monitoring
3. Camera Init	OV2640 configured; PSRAM allocated for frame buffer	Will halt if camera init fails — check connections

4. GPIO Setup	Motor, buzzer, TRIG, ECHO pins configured as outputs/inputs	Motor pins start LOW (stopped)
5. WiFi Connect	Connects to configured SSID; prints IP address to Serial	Must be on same network as control browser
6. Server Start	HTTP server begins listening on port 80	Registers /move, /stream, /status handlers
7. Streaming	Browser opens /stream; MJPEG frames pushed continuously	Frame rate depends on resolution and WiFi quality
8. Command Handling	Browser sends GET /move?dir=forward etc.	Obstacle check runs before forward is executed
9. Face Detection	Background task calls detectFace() every ~2 seconds	Uses a fresh frame capture; independent of stream
10. Buzzer Alert	If face detected, GPIO4 drives buzzer HIGH for 500 ms	Alert fires once per detection event

9. Setup Instructions

9.1 Arduino IDE Configuration

- Install the ESP32 board package via Boards Manager: https://dl.espressif.com/dl/package_esp32_index.json
- Select Board: AI Thinker ESP32-CAM
- Upload Speed: 115200 (lower if uploads are unreliable)
- Partition Scheme: Huge APP (3MB No OTA / 1MB SPIFFS) — required for face detection libraries
- Core Debug Level: None (or Error for production)
- PSRAM: Enabled

9.2 Upload Procedure

- Connect GPIO0 to GND before powering on the board to enter flash mode.
- Use a USB-to-Serial adapter (FTDI or CP2102) connected to U0TXD, U0RXD, GND, and 5V.
- After upload completes, disconnect GPIO0 from GND and press RESET.
- Open Serial Monitor at 115200 baud to verify startup and obtain the assigned IP address.

9.3 WiFi Configuration

```
const char* ssid     = "YOUR_WIFI_SSID";
const char* password = "YOUR_WIFI_PASSWORD";
```

Set these constants before compiling. The robot connects to the local network as a client and the assigned IP is printed to Serial on boot. Navigate to http://<IP_ADDRESS> in a browser on the same network.

> **Power Caution**
> Never power the ESP32-CAM from a USB adapter that also powers the motors. Motor current spikes can cause brownouts and unpredictable resets. Use a dedicated 5V supply with at least 2A capacity for the logic side, and a separate supply for motors if current draw is high.

10. Limitations

The following limitations are inherent to the current design and should be understood before deployment:

Limitation	Detail
Face detection latency	Detection runs on the ESP32's single-core AI accelerator and takes 200–600 ms per frame. This is not suitable for real-time tracking and may miss fast-moving subjects.
Stream stutter during detection	Running face detection simultaneously with MJPEG streaming competes for CPU time and the camera buffer. Frame rate may drop noticeably during detection cycles.
Limited processing power	The ESP32 is a microcontroller, not a CPU. It cannot run complex neural networks. Only lightweight detection models are feasible; recognition is out of scope.
HC-SR04 accuracy limitations	The HC-SR04 is only reliable between approximately 2 cm and 400 cm. It has a narrow detection cone (~15°) and may miss thin or angled objects. Walls and wide obstacles are detected reliably.
Power sensitivity	Motor noise can cause voltage sags on the 5V rail, leading to ESP32 brownout resets. This is particularly pronounced at startup and direction changes.
No encryption or authentication	The HTTP control interface is open on the local network. Any device on the same subnet can send commands. Add authentication if deploying in a sensitive environment.
L298N efficiency	The L298N has a voltage drop of approximately 2V across its H-bridge. At 7.4V battery input, motors see ~5.4V. Higher motor voltage requires a higher battery or a different driver.
Single-core camera bottleneck	The OV2640 and the ESP32's camera driver share a DMA channel. High-resolution streaming (1600×1200) is not compatible with simultaneous face detection — use QVGA or VGA for detection.

11. Future Improvements

The following enhancements are natural next steps for this platform:

- PWM Speed Control: Replace the fixed HIGH/LOW motor signals with PWM to allow variable speed, improving maneuverability and reducing current inrush.
- Dual Power Rails: Separate the motor supply from the logic supply with a proper DC/DC converter, eliminating motor noise on the ESP32 power line.
- OTA Firmware Updates: Implement over-the-air updates to allow firmware changes without physical access to the upload header.
- HTTPS / Authentication: Add basic HTTP authentication or switch to an HTTPS server to prevent unauthorized control on shared networks.
- Autonomous Patrol Mode: Implement a waypoint-following or reactive navigation mode that does not require continuous user input.
- Event Logging: Store face detection events with timestamps on the onboard microSD card for offline review.
- Mobile App Interface: Replace the browser control panel with a dedicated mobile application for lower latency and richer feedback.

12. Conclusion

The ESP32-CAM Security Robot demonstrates the viability of building a functional surveillance and intrusion-detection platform using a single low-cost microcontroller module. By integrating live video streaming, on-device face detection, motorized mobility, and ultrasonic obstacle avoidance into a cohesive system, the project covers a meaningful cross-section of embedded systems engineering disciplines.

The design is intentionally constrained — it runs entirely on local WiFi with no cloud dependency, no external compute hardware, and no proprietary software stack. Within those constraints, it delivers a working prototype that is suitable for further development, academic documentation, and portfolio demonstration.

The limitations identified in Section 10 are honest reflections of the platform's boundaries, not defects. They point clearly toward the next engineering challenges: power management, concurrency, and compute headroom — all tractable problems for the next revision.

Why This Project Is Valuable

Cross-Domain Engineering

This project integrates hardware interfacing, real-time firmware, computer vision, networking, and web UI into a single coherent system. Building it requires competence across all of these domains simultaneously — exactly the kind of work embedded engineers face in industry.

Practical Constraints, Real Solutions

Every design decision here — the voltage divider, the PSRAM allocation, the partition scheme, the power decoupling — reflects genuine engineering judgment, not textbook theory. The project shows the ability to read datasheets, trace failure modes, and make deliberate trade-offs.

Deployable Prototype

This is not a simulation or a hello-world LED blink. The robot moves, streams video, detects faces, avoids obstacles, and alerts the user — all in a package that can be powered on a table or sent across a room. That makes it a credible engineering artifact, not just an academic exercise.

Clear Path to Improvement

The documented limitations and future improvements show engineering maturity: knowing what the system cannot do, and why, is as important as building what it can. This self-awareness is a mark of professional engineering practice.

ESP32-CAM Security Robot — Technical Documentation

Document generated for GitHub / Portfolio / Internship Showcase purposes